腾讯WorkBuddy实践：如何把Agent做成可用产品

Anne 腾讯技术工程 2026年7月24日 02:36 广东 230人

我们总以为，把 Agent 做好，关键在于换一个更强的模型，或者把提示词写得更长、更细。

但真正进入一线场景后会发现，模型能力只是起点。Agent 能否稳定完成任务，还取决于它如何被引导，上下文如何被组织，工具和权限如何接入，结果如何被验证，以及 Harness 如何把一次次不确定的模型输出，转化为更稳定、可控的执行过程。

这篇文章邀请 WorkBuddy 团队策略产品经理 Anne，从产品视角拆解 Agent 的运行机制。她负责 WorkBuddy 研发与办公场景 AI Agent 的上下文策略设计与落地，是腾讯内部「上下文工程」方向最贴近一线的实践者之一。文章不堆叠概念，而是沿着“模型—上下文—Harness—Loop”的链路，讲清楚 WorkBuddy 如何把 Agent 从模型能力推进到可用产品能力。

从模型到Harness：
WorkBuddy如何把Agent做成可用产品

作者：Anne WorkBuddy策略产品经理

一个常见判断是“模型够强，剩下交给提示词”。但把 Agent 做成能在生产环境稳定完成任务的产品后，会发现模型只承担其中一部分：**工具接入、上下文组织、权限边界、结果验证、反馈纠正和跨会话延续**，都会直接影响产品是否可靠。

这篇文章分成两部分。前半部分面向还没有 Agent 基础的同学，先讲清大语言模型（LLM）、工具调用（Function call）、 系统提示词（System Prompt）、模型上下文协议（MCP）、技能 （Skill）和 插件（Plugin） 这些基础概念，说明模型为什么需要产品侧提供工具、上下文和执行环境。

后半部分回到 WorkBuddy 的产品实现，重点讨论 Context Engineering 和 Harness Engineering：WorkBuddy 如何选择和组织上下文，如何通过前馈、反馈、权限、验证、编排和可观测性，让 Agent 不只是能执行任务，而是能更稳定、更可控地完成任务。最后再简要讨论 Loop Engineering，说明这套机制如何进入长期任务循环。

阅读时可以带两个视角：作为**构建者**，看这些工程如何提高 Agent 执行任务的可靠性；作为**使用者**，看如何更有效地使用 AI Agent 产品。

一、先把模型看成一个无状态的函数

对产品侧而言，不需要展开 Transformer 的底层原理，只需要一个抽象：**模型是一个根据输入产生后续文字的函数。**

模型能力来自三个训练阶段：

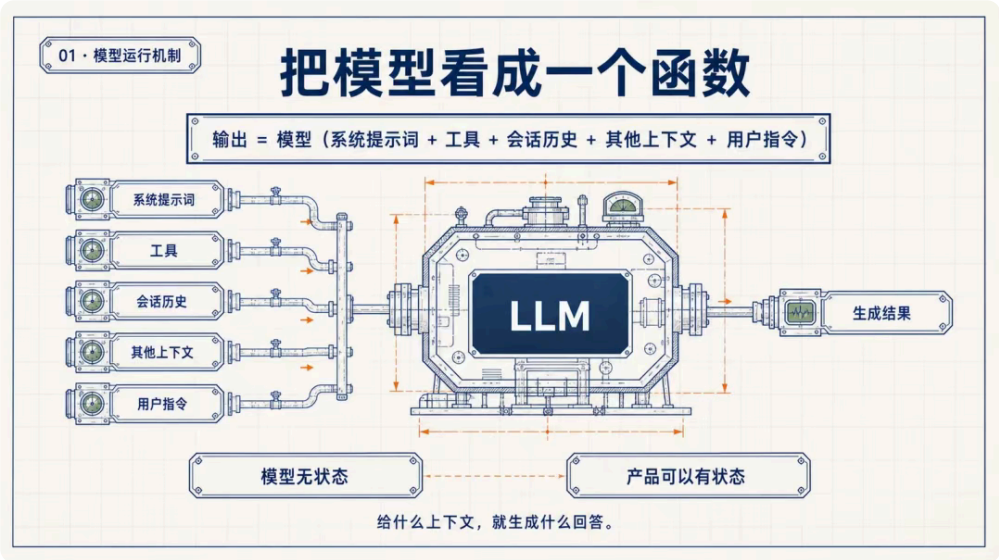
1. 预训练（pre-training）：模型在海量文本上反复执行“根据前文预测下一个 token”，习得语言、世界知识和部分推理能力。这一阶段的模型只能根据已有文本续写后续内容。

2. 后训练（post-training）：用问答、工具调用、安全边界等数据，把基座模型训练成能听指令的助手。这一阶段后，模型对“中国的首都是哪里”会回答“北京”。

3. 偏好优化与强化学习（Reinforcement-learning）：面对多个候选回答或操作路径时，用人类反馈和评分提高模型选中更有用、更正确选项的概率。对一个步骤组合很多的任务，训练时按结果质量、效率和规范给不同打分，反复之后模型收敛到更优的执行路径。

经过这些训练阶段后，我们得到的是一个具备大量基础知识、能够理解并遵循人类指令的模型。从产品运行机制看，一次模型调用可以类比为一个函数：

```
输出 = 模型 (系统提示词 + 工具 + 会话历史 + 其他上下文 + 用户指令)
```



图：模型调用抽象

这个抽象包含两条约束，决定了上层所有工程的存在理由：

1. 模型是无状态的。它不会自动保留上一次调用的内容。模型虽然无状态，但产品可以有状态。对话历史、Memory、数据库由产品在模型外部保存，需要时再放进本次输入。

WorkBuddy 的对话连续性、记忆和工作进度，都由产品侧维护状态再注入实现，模型本身不承担存储。

2. 模型的知识截止到训练日期。训练之后发生的事，模型默认不掌握。询问训练日期之后的实时信息时，模型无法回答，需要先用工具查询再放进上下文。

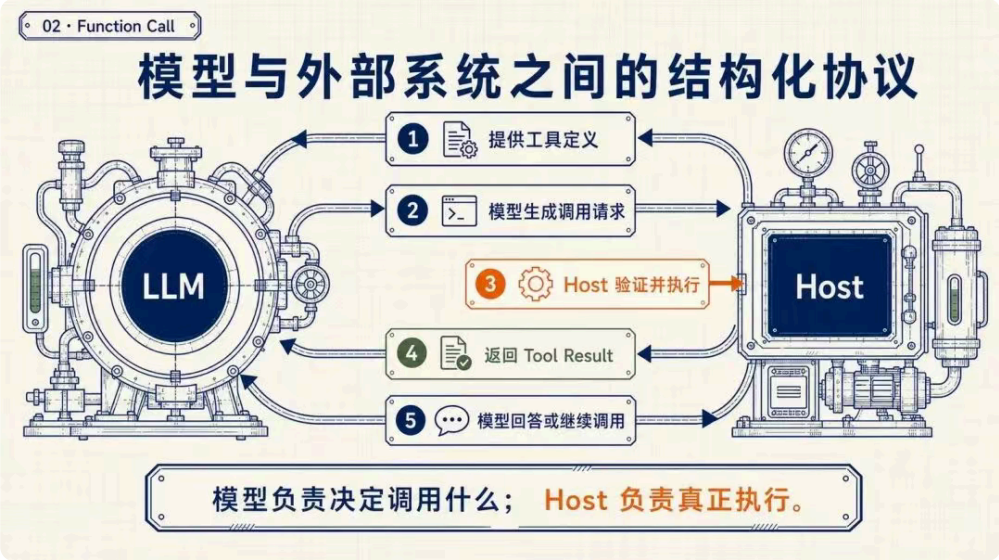
也就是说，模型本身提供的是语言理解、推理和生成能力；但像“当前世界杯赛况”这类实时信息，或者读文件、查数据库这类外部动作，都不是模型自带的能力。产品需要在模型外接入工具和执行环境，让模型知道有哪些工具可用，并把执行结果回传给模型。

这就引出了 Agent 的基础机制：工具调用。

二、用户能感知到的四个概念：工具调用 / MCP / Skill / Plugin

工具调用：模型怎么请求执行动作

工具调用（也常叫 function call、**Tool Call**）是模型与外部系统之间的结构化协议：**模型负责生成调用请求，Agent 负责执行。**



图：工具调用流程

完整流程：

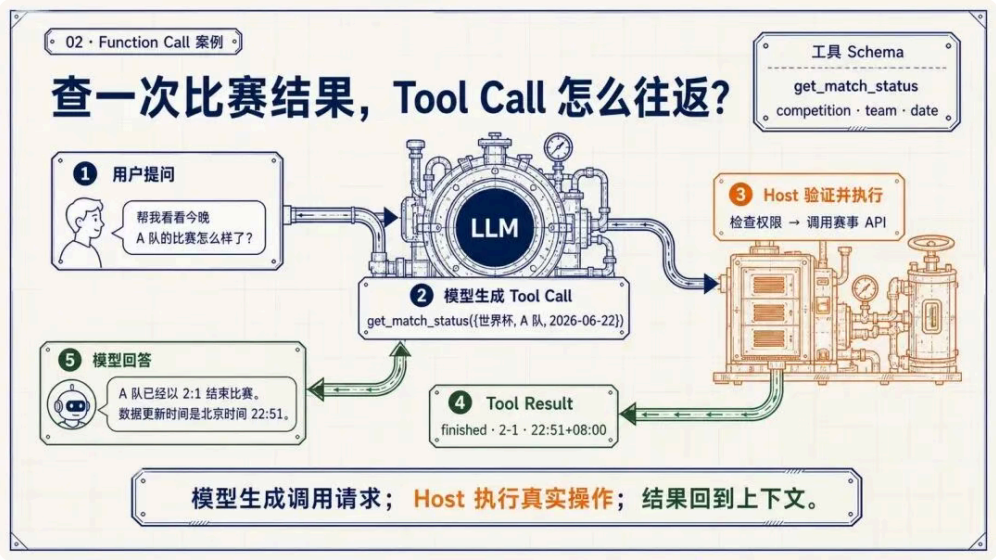
1. 产品把可用工具的名称、用途和参数 Schema 提供给模型；
2. 模型根据用户目标，输出一个结构化的调用请求；
3. Agent 校验参数、检查权限，执行 API、脚本或本地函数；
4. Agent 把执行结果作为 tool result 放回上下文；
5. 模型读取结果，决定直接回答还是继续调用其他工具。

一个示例工具定义如下，它作为上下文提供给模型：

```
{
```

```
  "type": "function",
  "name": "get_match_status",
  "description": "查询指定赛事在指定日期的实时或最终赛况。用户询问比分、开赛时间、
  "parameters": {
    "type": "object",
    "properties": {
      "competition": { "type": "string", "description": "赛事名称，例如世
      "team":        { "type": "string", "description": "球队名称" },
      "date":        { "type": "string", "description": "YYYY-MM-DD；未
    },
    "required": ["competition", "team"],
    "additionalProperties": false
  }
}
```

下图展示了一次工具调用的完整执行过程：用户提出查询请求后，模型根据工具定义生成工具调用；Agent 收到请求后校验参数和权限，再去获取实时的数据；工具返回结果后，模型读取工具执行结果，并基于结果生成最终回答。

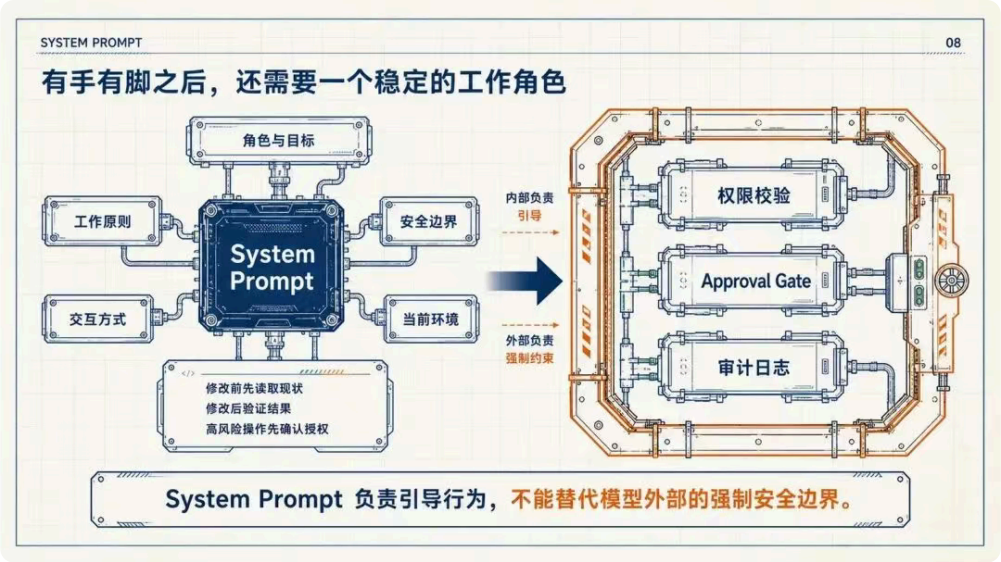


图：工具执行过程示例

这里有一个容易被忽略的要点：**持有 API Key、发起请求、修改数据的是 Agent，不是模型**。模型负责生成调用请求，Agent 负责执行外部操作。因此，权限、审批、参数校验和审计日志都必须由模型外部的工程机制执行。把这些校验放在 Agent 执行层，是高风险操作能被拦下的前提。

System Prompt：给 Agent 一个稳定的工作角色

有了工具调用，Agent 具备执行能力，但还缺少稳定的工作角色。同一个模型既能写诗也能改代码、查数据；WorkBuddy 需要它在每次运行中都明确：自己是什么产品、能做什么、按什么原则工作、什么情况必须停下来询问用户。



图：System Prompt 的作用

System Prompt 定义当前产品和本次运行的高优先级工作契约，通常包含：

- **角色与目标**：你是一个能读写文件、运行命令、操作浏览器的工作助手。
- **能力地图**：哪些工具可用，何时查资料、读文件、验证结果。
- **工作原则**：先理解目标再执行；长任务先拆分；修改后要测试；不确定时不猜。
- **安全与权限边界**：删除、发送、支付、发布等高风险动作需要审批。
- **交互风格**：使用用户选择的语言，进度更新简短，最终结论清晰。
- **当前环境**：操作系统、Shell、时间、工作目录等运行时信息。

```
1 你是 WorkBuddy。
2 你可以使用文件、Shell、浏览器和已连接的业务工具完成任务。
3 修改前先读取现状；修改后用可观察的方式验证。
4 发送、发布、删除或其他不可轻易撤销的操作，执行前确认用户授权。
```

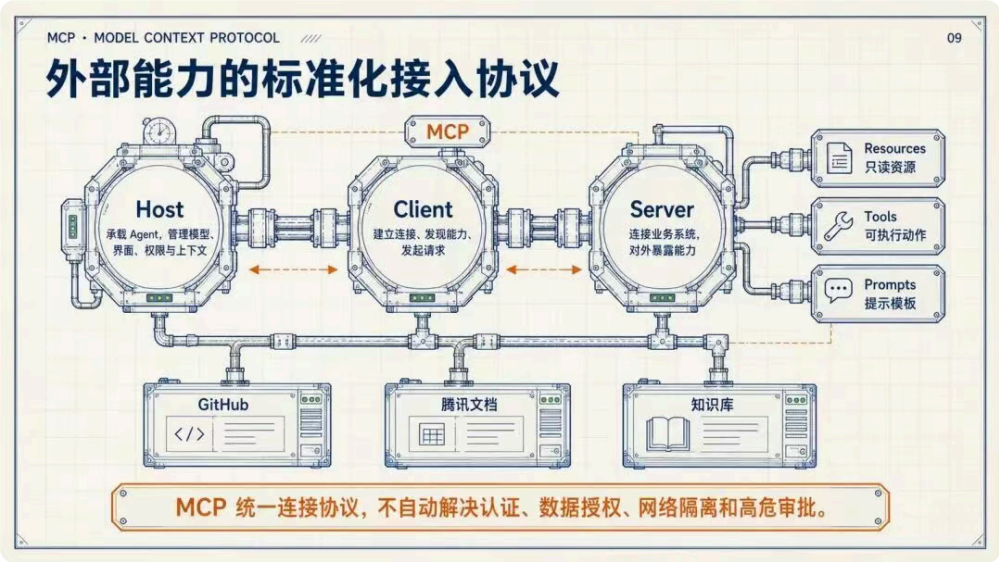
System Prompt 只能引导，不能强制。权限校验、Sandbox、Approval Gate、审计仍由模型外部的系统执行。这个区别在 Harness 章节会反复出现。System Prompt 也不该承载所有信息，WorkBuddy 采用的分层是：所有任务都适用的角色与安全要求放 System Prompt，项目规范放 Workspace 规则文件，某类任务的步骤放 Skill，当前请求和进度作为动态上下文按需加入。

模型上下文协议（MCP）：外部系统怎么标准化接入

到这里已经有了一个具备基础能力的 Agent，它能读写文件、运行脚本、执行命令等。但是用户在使用一个 Agent 产品的时候往往需要访问很多外部系统，例如读写 GitHub Issue/PR、访问外部文档、知识库、网盘等等。如果每接入一个系统都单独适配它的认证、接口、参数和返回格式，Agent 产品会很快变成一堆专用集成，维护成本很高。

MCP 试图解决的是外部能力接入的标准化问题。

Anthropic 在 2024 年底发布了开放协议 **Model Context Protocol（MCP）**，用统一方式连接 AI 应用和外部数据源、工具。它为 AI 应用接入外部能力提供统一接口，Agent 不需要分别适配每个系统的调用方式。



图：MCP 统一接入

多数用户对 MCP 的直接感知是“Agent 多了一批可调用的工具”。但它的名字是 Model Context Protocol 而不是 Model Tools Protocol，因为它提供的内容不限于工具。MCP Server 向 Agent 提供三种原语，关键差异在于谁来驱动：

- **Resources（资源）**：有 URI 标识、可读取的只读内容（一个文件、一条数据库记录、一段实时数据）。它是应用 / Agent 驱动的——由 Agent 或用户决定何时读取、注入，本职用法是把读出的内容直接拼进 messages，不经过 tool。

- **Tools（工具）**：模型能调用的动作 / 函数。它是模型驱动的——模型在推理时自己决定是否调用。结果通常以 text 回流进上下文供模型继续推理，但也可以把 structuredContent 分流给 UI 而不进上下文。

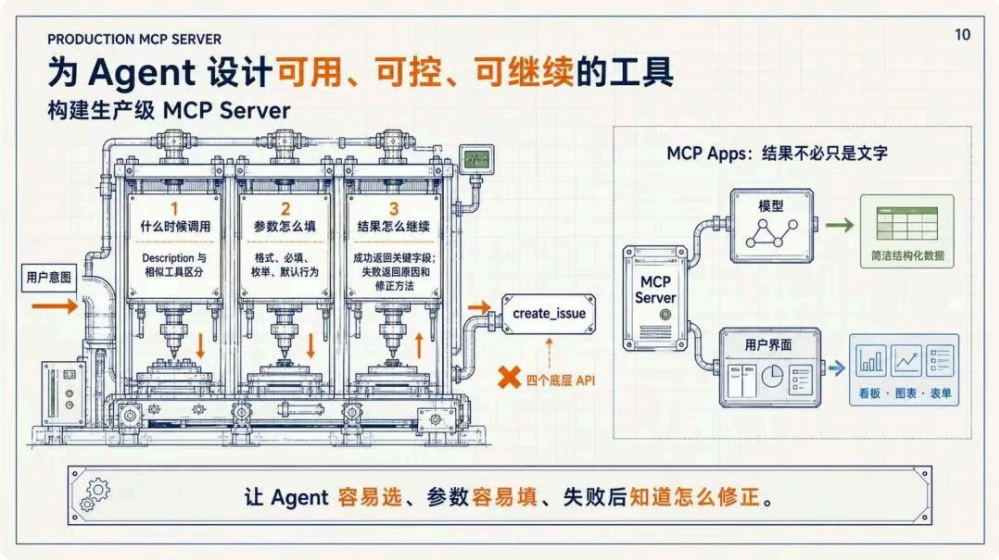
- **Prompts（提示模板）**：Server 预先组织好、可复用的一组消息。它是用户驱动的——由用户主动点选（如斜杠命令）触发，模板内部还能引用 resource，把指令和文件内容打包带入。

这三类原语说明，MCP 处理的不是单一的“工具调用”，而是 Agent 与外部系统之间的信息、动作和提示模板如何被组织和传递。进一步看，外部系统返回的内容也不一定都要进入模型上下文：有些内容适合给模型阅读，有些内容更适合直接展示给用户，或者让用户在界面上确认和操作。

2026 年发布的官方扩展 MCP Apps 就是在这个方向上的延伸。它允许工具返回可直接渲染在对话中的交互式 UI，例如看板、图表、表单和确认界面。面向模型的摘要继续进入上

下文，面向用户的界面数据直接交给 UI 渲染，不必全部塞进模型上下文。这样既保留了交互体验，也减少了上下文占用。

运行结构上，MCP 有三个角色：承载 Agent 的产品（如 WorkBuddy）、负责建连和发请求的 MCP Client、对外暴露能力的 MCP Server。MCP 统一的是连接协议，Server 背后仍可以是 REST、数据库、SDK，Agent 不需要理解这些差异。



图：MCP 运行结构

关于 MCP Server，还有一个关键设计原则：**按用户意图组织工具，不照搬底层 API。**例如“创建 Issue”可能涉及创建、加描述、加 tag、加附件四个底层接口，但对 Agent 应该只暴露一个 `create_issue` 工具，把描述、tag、附件作为参数。Issue 相关操作也可以收进一个工具，用不同 action（create / delete / update / close）区分。一个可用的工具至少要说明三件事：什么时候调用、参数怎么填、结果怎么继续处理。

Karpathy 在《Software Is Changing (Again)》里指出，Agent 是一类新的数字信息消费者和操作者。过去是人通过 GUI、程序通过 API 使用软件，现在多了一类介于两者之间的使用者。因此做产品时除了“人怎么点”，还要考虑“Agent 怎么理解、怎么操作、怎么验证”。

> *“Can we just build for agents?”（我们能只为 Agent 开发软件吗？）*

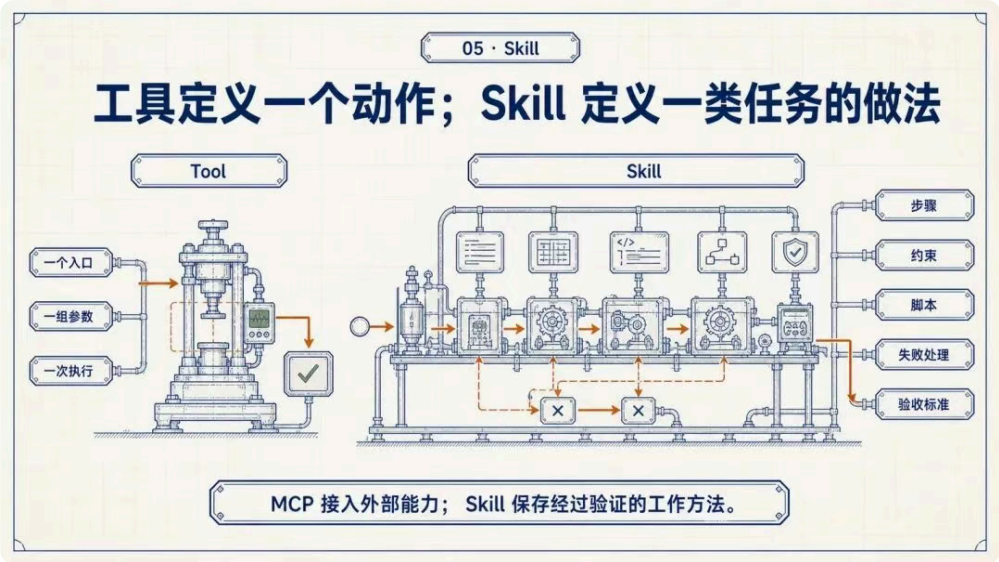
Skill：一类任务该按什么流程做

有了 MCP 和 Tool，Agent 已经能调用很多外部能力。但真实任务通常不是调用一次工具就结束。所以还需要 skill。

以在某个仓库提交 PR 为例，它不止调一次 API——还要读仓库规则、执行测试、生成变更说明、处理失败，最后才创建。Skill 把这些经过验证的工作方法保存下来，通常包含说明、步骤、脚本、命令和判断标准。模型处理匹配任务时先读 Skill，再按流程执行：

```
适用场景：用户要求为当前仓库的代码变更创建 PR。
流程：
1. 读取 AGENTS.md / WORKBUDDY.md 和仓库贡献规范。
2. 检查 git status，不覆盖用户的未提交修改。
3. 阅读 diff，识别变更范围与风险。
4. 运行与改动相关的格式化、类型检查和测试。
5. 仅在验证通过后生成标题和描述，如实列出未验证项。
6. 确认用户已授权发布，再 push 并创建 PR。
完成标准：
- PR 只包含本任务的变更。
- 已记录所运行的测试与结果。
- 标题、正文、Issue 关联符合团队规范。
```

Tool 负责“一个动作”，Skill 负责“一类任务的做法”。



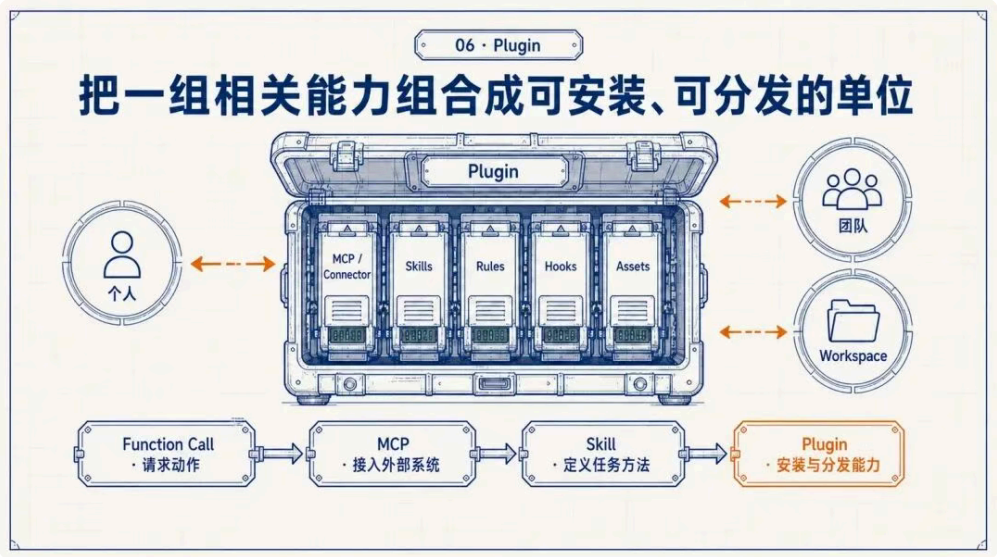
图：Skill 的作用

一句话区分：**MCP 解决“外部系统怎么接入”，Skill 解决“这类任务应该怎么做”。**两者可以组合——一个“发周报”Skill 可能同时调用腾讯文档 MCP、知识库 MCP 和本地脚本。Skill 里还要写清失败分支：测试失败如何判断、没有权限就停在草稿。

Plugin：一组能力怎么打包分发

MCP 负责连接外部系统，Skill 负责保存任务流程。但真实场景里，一个完整能力往往不只包含单个工具或单个流程，而是一组连接、流程、规则、Hooks 和模板。

Plugin 解决的是“能力组合如何安装和分发”的问题。它**把多种相关能力组合成可安装、可分发的单位。**



图：Plugin 打包

例如一个团队要接入内部研发流程，可能同时需要 MCP（连接代码仓库、Issue、流水线、文档）、Skills（提 PR、查构建、写变更说明）、Rules（代码 / 分支 / 安全规范）、Hooks（提交 / 编辑 / 执行命令前后检查）。Plugin 把这些组合起来，支持按团队、项目或个人作用域安装：

```
Plugin: team-dev-workflow
├─ MCP: 读写 Issue、MR、构建结果和内部文档
├─ Skills: /issue-start、/create-pr、/debug-ci、/release
├─ Rules: 分支、Commit、安全、架构规范
├─ Hooks: 编辑后 lint，提交前测试，发布前审批
└─ Assets / Templates: PR 模板、发布说明、架构图
```

注意：**MCP 是跨产品协议，Plugin 是产品层的打包概念。**不同 Agent 产品对 Plugin 的内容、安装方式和作用域可以有不同定义，所以它是通用产品概念，但不属于通用标准。

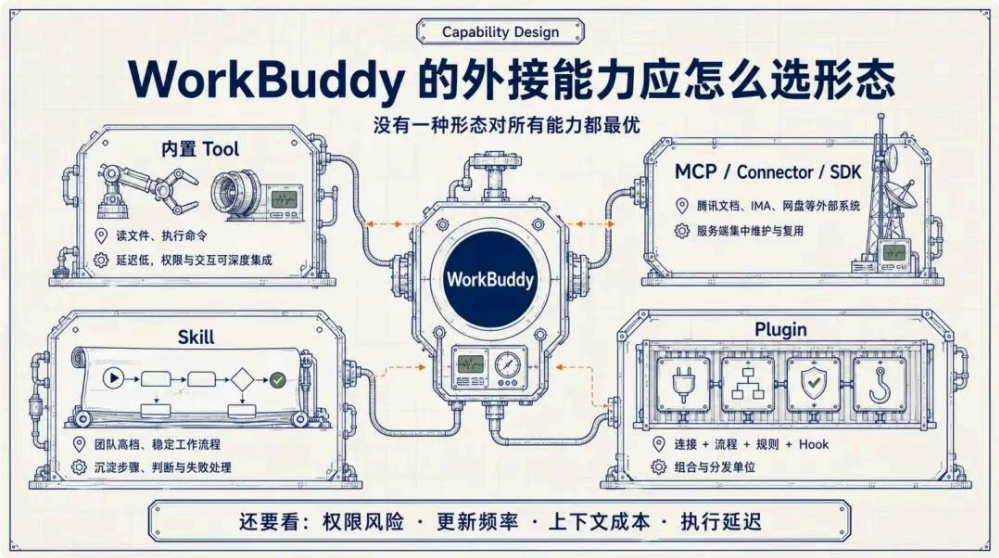
四个概念的关系可以一表概括：

概念	核心问题	主要消费者	典型内容
工具调用	一个模型怎么请求执行动作？	模型 + Agent	名称、描述、Schema、调用结果
MCP	外部系统怎么标准化接入 Agent？	Agent / Server	Tools、Resources、Prompts
Skill	一类任务应该按什么方法做？	Agent	流程、约束、脚本、验收标准
Plugin	怎么把一组能力安装和分发？	用户 / 团队 / 产品	MCP、Skills、Rules、模板

Agent 外接能力怎么选形态

前面讲的 Function Call、MCP、Skill 和 Plugin，解决的是不同层次的能力组织问题。Function Call 是模型请求动作的基础协议；MCP / Connector 负责把外部系统标准化接入；Skill 沉淀一类任务的执行流程；Plugin 则把连接、流程、规则和模板组合成可安装的能力包。

因此，设计一个外接能力时，不能只问“能不能接进来”，还要判断它更适合哪种产品形态。不同能力的边界、更新频率、权限风险、上下文成本和复用方式不同，适合的接入方式也不同。



图：外接能力形态选择

需求	优先形态	原因
稳定的底层操作，如读文件、执行命令	内置 Tool	延迟低、权限和 UX 可深度集成
腾讯文档、IMA、乐享、网盘等外部系统	Skill / MCP / Plugin	服务端集中维护，能力可标准化复用
团队高频、稳定的工作流程	Skill	沉淀步骤、判断标准与失败处理
需要同时装连接、流程、规则和 Hook	Plugin	作为组合与分发单位

没有一种形态对所有能力都最优。判断标准是：能力边界、更新频率、权限风险、上下文成本、执行延迟与跨产品复用价值。

三、全景视图：一次完整任务长什么样

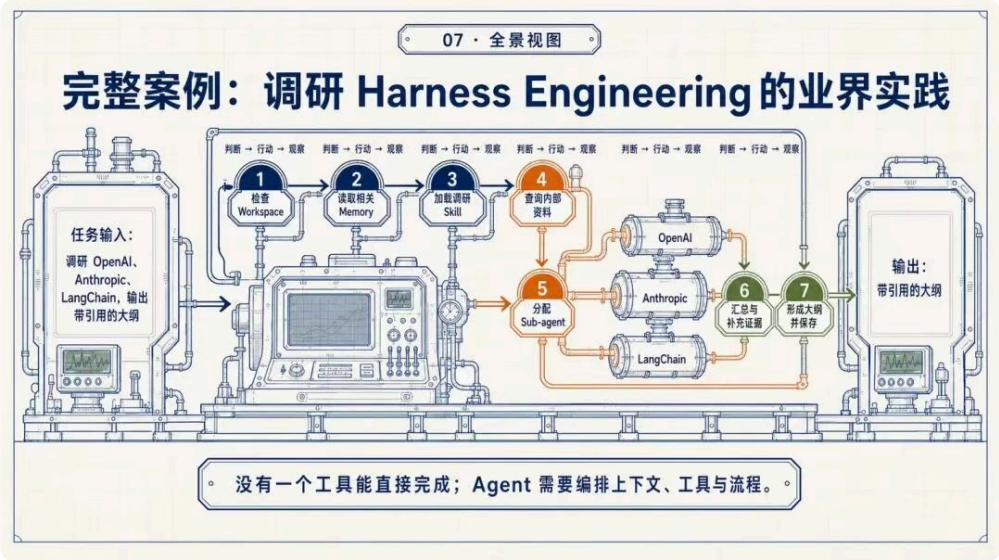
把前面的概念串联起来。一次完整任务里，用户目标、系统指令、对话历史、当前环境和相关记忆一起进入模型；模型决定下一步，通过 Function Call 请求内置 Tools、MCP 或其他能力；Agent 执行、验权和审批，再把 Tool Result 返回给模型。

以一个真实任务为例：用户要求“调研 OpenAI、Anthropic、LangChain 在 Harness Engineering 上的实践，整理核心观点、解决的问题和对我们团队的借鉴，输出一份带引

用的大纲”。

WorkBuddy 大致会进行以下步骤：

> → 检查已有资料
> → 读取 Memory 和 Skill
> → 查询内部问题
> → 分配 Sub-agent 并行调研
> → 汇总观点和证据
> → 补充缺口
> → 生成大纲



图：一次完整任务的信息流

其中几个动作由 WorkBuddy 的产品机制支撑：

第一，查看当前 Workspace。Agent 不会立即开始网页搜索，而是先检查当前工作区是否已有相关资料、草稿、PDF、网页书签或分享模板，也会读取当前 Workspace 的规则，确认文件保存位置、格式和命名方式。这可以避免重复调研，并让输出延续已有工作。

第二，读取与任务相关的 Memory。Agent 会读取当前任务相关的记忆，例如用户偏好的表达方式、文章结构，以及是否需要区分构建者和使用者视角。Memory 不替代事实判断，但会影响内容组织和表达方式。

第三，查找适用的 Skill 和规则。“帮我调研”不是简单问答，而是一类有流程和完成标准的任务。调研类 Skill 会要求 Agent 拆分调研对象、优先查找一手资料、记录来源，并区分原作者观点和自己的推论。

第四，连接内部数据源。为了让结论能落回团队实际问题，Agent 会结合内部资料，例如产品架构文档、常见问题、已有规则和工具列表。

第五，拆分调研任务并分配给 Sub-agent。OpenAI、Anthropic、LangChain 的资料可以分别交给不同 Sub-agent 处理。主 Agent 为每个 Sub-agent 提供统一输出格式，但只提供各自任务所需的上下文，减少信息干扰。

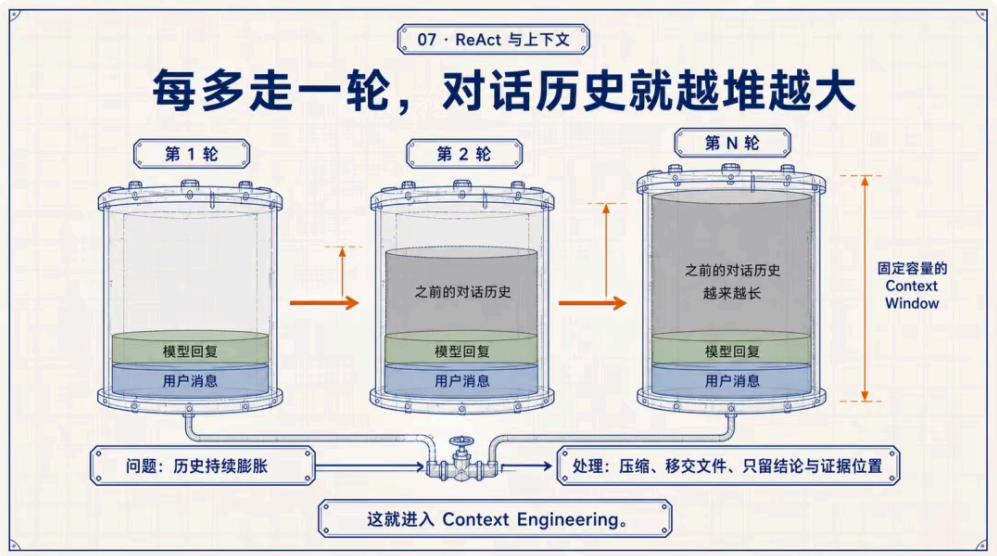
第六，Sub-agent 阅读资料并返回结果。每个 Sub-agent 使用适合的工具获取信息，并返回结论、证据、来源和适用范围。例如 OpenAI 侧重代码质量约束，Anthropic 侧重长任务状态管理，LangChain 侧重运行环境和编排逻辑。

第七，主 Agent 汇总并补齐缺口。主 Agent 合并材料、去重、识别冲突，并把零散实践放进统一框架中。

第八，生成大纲并保存任务状态。Agent 按 Skill 和 Workspace 规则生成大纲草稿，保存到合适位置，并记录资料来源、未解决问题和下一步建议。

这个案例展示的过程由多轮的“工具调用---拿到工具结果----决定下一步操作”的循环，就是 ReAct 循环（reasoning - acting）：“判断—行动—观察”。

整个过程中，每次观察到的结果都被放到了模型的上下文里，**主上下文的长度逐渐增大。**所以我们必须要进行上下文管理，也就是 context engineering。

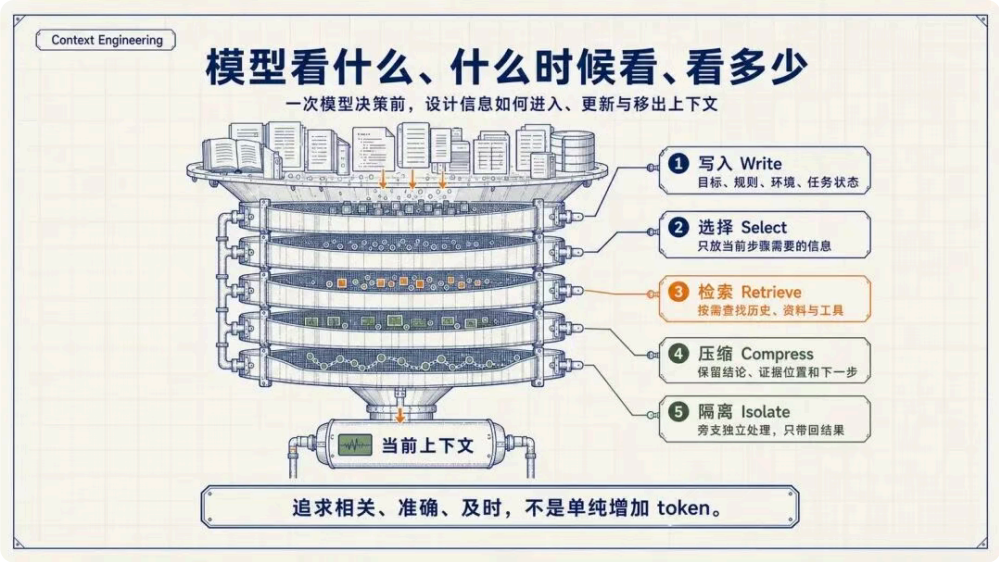


图：ReAct 多轮循环

四、Context Engineering：模型这一刻该看到什么

我们可以把 Context Engineering 定义为：

> *在一次模型决策前，设计哪些信息进入上下文、以什么形式进入、放在什么位置、何时更新或移出，以提高模型做出正确下一步决策的概率。*



图：Context Engineering 的五类动作

它包含五类动作，注意每个动作的精确含义：

1. 写入（Write）：把目标、规则、环境和任务状态显式写进上下文，别让模型靠猜；

2. 选择（Select）：从已在手的候选信息里，只挑当前这一步需要的放进窗口（这是 filter）；

3. 检索（Retrieve）：当前不在手的信息，从历史会话、资料库、工具目录里按需捞进来（这是 pull）；

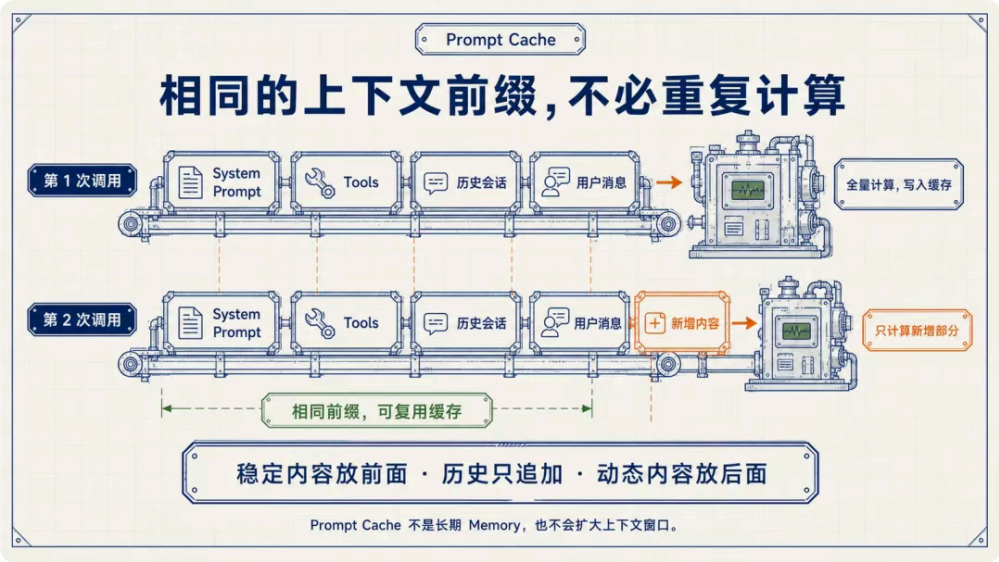
4. 压缩（Compress）：长内容外置到文件、只留结论与证据位置，同时清理窗口里过期或重复的内容；

5. 隔离（Isolate）：用独立会话或 Sub-agent 处理旁支任务，只把结果带回主线，避免污染主上下文。

产品需要把相关环境信息组织成 context，例如：操作系统是 Windows / Mac / Linux、Shell 是 bash 还是 powershell、用户当前时间时区地理位置（模型不感知时间）、当前打开的文件、产品语言、当前仓库规则和未完成任务、已安装的 Skills 和已连接的 Connectors。WorkBuddy 在每次请求前组织这些信息，是 Agent 给出贴合当前环境结果的前提。

一个常见误区是“上下文窗口很大，全部放进去”。无关信息既占成本，也降低模型对当前重点的判断准确度。**Context Engineering 追求相关、准确、及时，不是单纯堆 token。**

Prompt Cache：上下文管理的第一要义



图：Prompt Cache 前缀复用

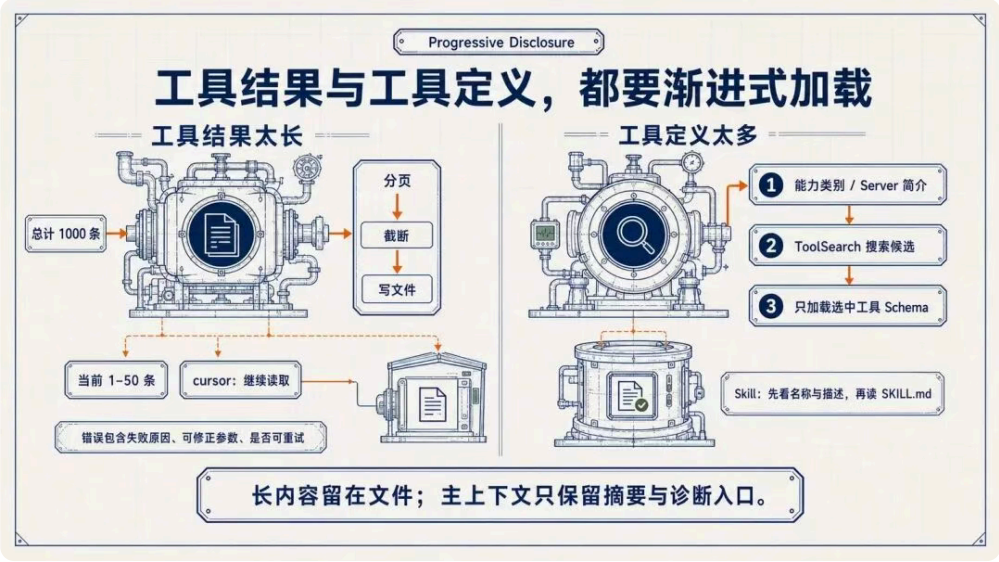
我们知道，多轮对话每次都带上之前的上下文，第二轮通常包含第一轮全部内容，再在末尾追加。每轮全量计算成本过高，每次重新计算显然不合理，所以模型厂商会缓存已计算过的前缀，只计算新增部分，这就是 Prompt Cache。

它按前缀匹配，所以 WorkBuddy 在上下文组织上遵循几条规则：

> ·System Prompt、基础工具定义、长期规则放前面，保持内容与顺序稳定；
> ·对话历史采用追加方式保存，不修改已发送过的消息；
> ·当前文件、任务进度、时间、工具结果、新加载的 Skill 等动态内容追加到后面；
> ·工具和 Skill 按需加载，避免每轮重新生成并排列完整能力列表；
> ·只在上下文过长需要压缩、或纠正错误信息时，才接受前缀变化和缓存重算。

随着用户对积分（成本）越来越敏感，缓存命中率正在成为被普遍关注的工程指标。

渐进式加载：长结果和大工具集



图：渐进式加载

我们在 function call 的章节有讲到，我们需要把工具定义放入上下文，模型才能调用工具。随着 Agent 能力越来越多，工具数量也不可避免的越来越多。因此，工具定义部分也需要做上下文管理：哪些工具默认暴露，哪些工具按需加载，工具结果过长时如何处理，都会影响 Agent 的执行效果。

工具结果过长时，WorkBuddy 给每个 Tool Result 设截断策略，超出时分页、截断或写入文件。截断时明确告诉模型“结果未完整”，附上总量、截断位置和继续读取方法，否则模型会把前 100 条误当成全部。错误也不只返回 error 或一段堆栈，还要返回失败原因、可修正参数、是否可重试和建议下一步。

工具定义过多时，每个 Tool Schema 都占上下文，工具越多、语义越重叠，模型选择越困难。WorkBuddy 采用分阶段的能力发现机制：先让模型看到工具名称和简要描述，再根据任务需要加载更具体的工具说明，同时也提供了工具检索能力。这样可以减少上下文占用，也降低工具过多带来的选择干扰。

Skill 也用同样机制，先看名称和描述，确认适用后再读完整 `SKILL.md`，需要时再打开参考资料与脚本。

高质量的 MCP 同样可以把要用的资源拉取到本地、提供 search 能力，减少上下文注入。

几乎所有外接能力都可以按这个思路组织：默认只暴露名称和简介，真正进入某个任务时再加载完整内容。这样上下文里始终只保留当前需要的能力，而不是把所有能力一次性铺开。

意图识别是这一思路的前置环节。用户的一句话背后可能对应问答、改代码、检索资料或调用外部系统等不同任务。系统先识别意图，再据此决定加载哪一类工具、Skill 和 MCP，让后续上下文只包含与当前任务相关的能力。意图识别负责“先选对方向”，渐进式加载负责“再按需展开”，两者共同控制上下文规模。

五、Memory：让正确的过去在正确的时候重现

记忆功能常被产品宣传为“越用越懂你”。更准确地说，它解决的是重复交代背景的问题：用户长期使用某个 AI 产品后，一些背景和历史不希望反复交代。模型本身没有长期记忆，产品要做的是从历史交互中提取少量可信信息，用于理解用户和延续任务。

和记忆相关的内容有三类：**聊天历史**（事情发生过，但不一定该影响未来，可作为 RAG 检索源）、**当前工作空间的工作记忆**（项目进度）、**长期记忆 Memory**（在用户发当前任务之前，默认就能代入上下文、影响结果走向的内容）。关键在于：Memory 系统要做一次**准入判断**——哪些历史信息可以继续影响未来要做的任务。WorkBuddy 把这次准入判断作为记忆系统的核心环节，控制哪些信息有资格进入后续上下文。

WorkBuddy 长期 Memory 的五类记忆

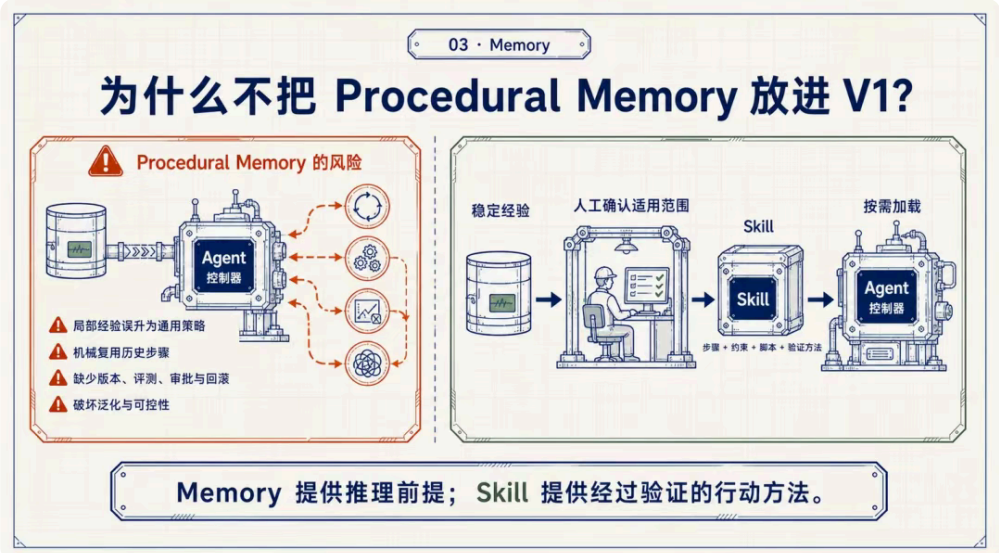
这里有一个容易混淆的点：**记忆类型回答“存什么”，作用域回答“在哪里生效”，两者是正交的两个维度。**WorkBuddy 会把长期信息拆成几类：

信息类型	存的是什么	在系统中的作用	例子
稳定事实	去情境化的稳定事实、长期偏好、已确认的默认假设	作为长期推理前提，减少重复获取用户基础信息	用户所在城市、常用工作语言
用户知识背景	用户的专业背景、知识水平和熟悉领域	调节解释深度和术语密度，不改变事实结论	用户熟悉 Context Engineering
行为信号	从多次真实交互观察到的稳定使用模式	作为交互策略的调节信号	回答前先查看当前工作空间
表达偏好	用户对表达方式的稳定偏好	控制"怎么说"，不影响"事实是什么"	先给结论、减少空话
会话延续信息	当前会话中仍有价值的目标、决策、进度、未完成项	帮助延续讨论和任务	已完成什么、下一步做什么

五类记忆的影响范围不同：

- Semantic 可作为后续默认前提；
- Style 只影响表达，不改变事实结论；
- Behavior Signal 要比用户明确表达的偏好更谨慎。用户偶尔要求一次“先看文件”，不代表以后所有任务都必须这样，WorkBuddy 需要观察多次稳定行为，并允许用户查看和纠正。

为什么没有把 Procedural Memory 放进长期记忆



图：陈述性记忆 vs 程序性记忆

以上五类记忆都属于陈述性记忆（declarative memory），记录的是“用户是谁、了解什么、发生过什么”，也就是用户特征、知识背景和历史交互。它们提供的是推理前提，不直接规定 Agent 该怎么做事。WorkBuddy 没有把 Procedural Memory（程序性记忆）纳入长期记忆，是因为程序性记忆记录的是“做事方法”，这类内容一旦作为长期记忆注入，会直接影响 Agent 的推理路径和执行顺序。

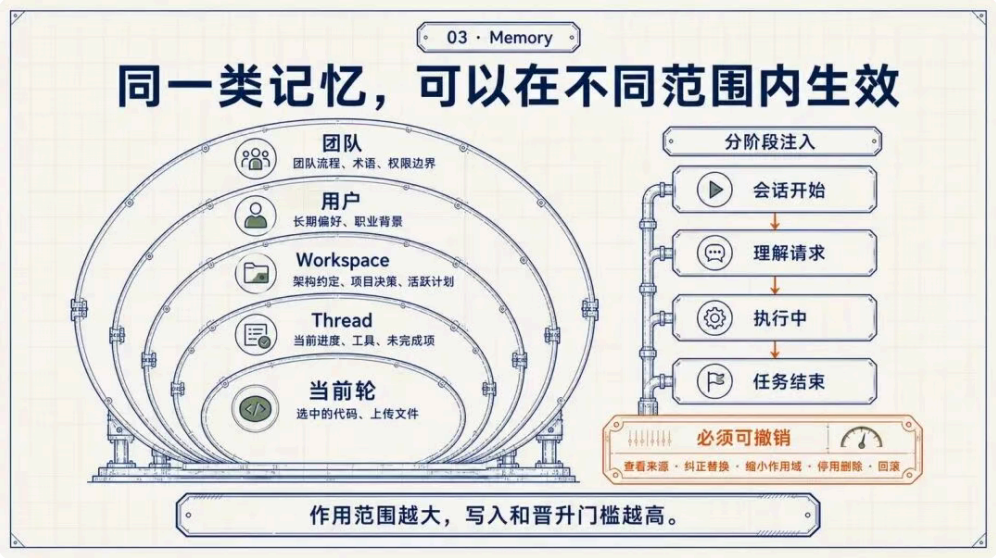
例如“用户熟悉 Python”是陈述性信息；“遇到所有调试任务，都先重启服务再查日志”是程序性策略。后者注入上下文后会带来几个问题：

- **局部经验被误升为通用策略**：一次任务里有效的步骤，换个项目未必成立；
- **干扰模型推理**：模型可能不再根据当前证据选路径，而是复用历史步骤，陷入局部最优；
- **隐性改写 Agent 行为**：程序性内容实际上接近动态 System Prompt，却可能缺少版本、评测、审批和回滚；
- **降低泛化与可控性**：如果 Agent 从每次任务自动学工作流，又没有稳定的奖励信号和 Eval，系统很难判断它是在改进还是在积累偏见。

所以 WorkBuddy 的选择是：**用户事实和历史状态放进 Memory，经过验证的工作方法保存为 Skill**——Skill 可版本化、可评审、可测试、可回滚、按需加载。

```
成功经验 / 稳定流程
        │ 人工提炼、明确适用范围和完成标准
        ▼
Skill（步骤 + 约束 + 脚本 + 验证方法）
        │ 版本化、可评审、可测试、可回滚、按需加载
        ▼
在匹配任务中指导 Agent 行动
```

Memory 的作用域分层与注入时机



图：记忆作用域分层

同一类记忆可以在不同范围内生效，作用范围越大、影响越高，写入和晋升的门槛也应越高：

层级	例子	解决的问题	典型失效方式
当前轮临时上下文	刚选中的代码、本次上传的文件	理解当前一步	任务结束即失效
会话 / Thread 记忆	做到哪一步、哪些工具已调用	让多轮对话连续	Thread 结束或任务完成后归档
项目 / Workspace 记忆	架构约定、活跃计划、已有决策	让 Agent 在某场景里接着做	项目变更、决策被替换或切换
用户级记忆	长期表达偏好、职业背景	减少反复自我介绍	用户修改、撤销或随时间降权
团队 / 组织记忆	团队流程、术语、组织边界	促进多人与多 Agent 协作	权限变更、制度更新

记忆的注入分阶段进行：冷启动时只注入少量高置信、高相关的人与项目摘要；请求理解时根据 query 激活候选 memory cards（仍保留来源和置信，不当成确定前提）；执行中需要证据时再回查原始会话或文件；任务收尾时从结果和用户纠正中提取候选记忆，做去重、冲突检查和作用域判定。

一个成熟的 Memory 系统需要同时支持写入、来源查看、用户纠正、冲突替换、时间衰减、降权、删除、回滚和临时停用。WorkBuddy 的 Memory 设计目标是：

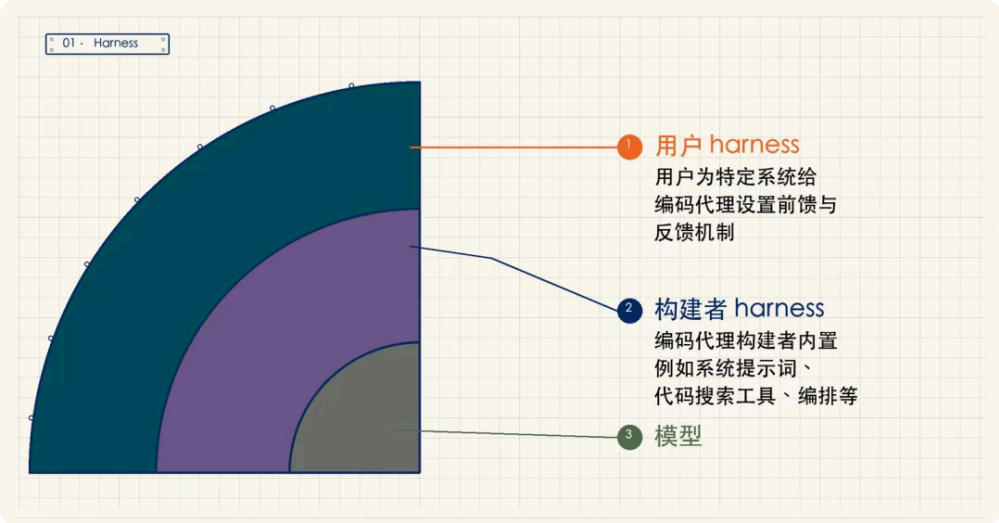
> ***让正确的过去，在正确的时候，以正确的作用域，正确的方式重新出现。***

六、Harness Engineering：引导、约束与整合

前面几节——上下文工程、MCP、Skills、记忆、压缩——都在解决一个问题：**Agent 是否获得了足够的信息。**这些机制只决定 Agent 知道得够不够。当 Agent 开始写文件、运行命令、操作外部系统时，会出现另一组问题：

- 执行方向是否正确？偏离后能否纠正？（方向）
- 哪些操作不允许执行？误删文件由谁拦截？（安全）
- 这些能力如何组织成一个能稳定运行的系统？（编排）

这些是 Harness Engineering 要解决的问题。Harness 可以按构建者和使用者分成两层：

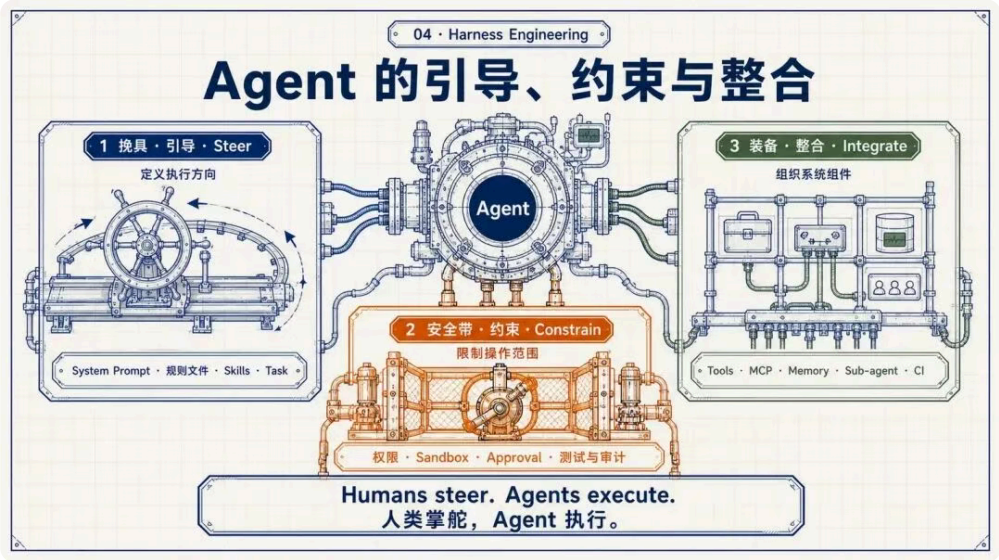


图：Harness 的两层同心圆

三个同心圆：核心是模型，外一圈是 Agent 构建者的 harness（System Prompt、代码搜索工具、编排等），最外圈是 Agent 使用者的 harness（针对自己系统配置的前馈和反馈控制）。**构建者通过产品机制控制模型行为，使用者通过产品提供的能力控制 Agent 和模型。**

三类能力：驾驭、约束、整合

Harness 一词原指套在马身上的整套装备。从词源出发可以拆出三类能力，对应 Harness 要解决的三个方面：



图：驾驭 / 约束 / 整合

1. 驾驭（Steer）。控制执行方向、速度和停止时机。对应到 Agent：System Prompt / 规则文件（`WORKBUDDY.md`、`AGENTS.md`）说明工作方式，Skills 规定某类任务的步骤，Task / Todo 把大目标拆成清单，错误消息中的自我纠正提示在卡住时提供方向（而非只给错误码），针对不同模型的微调处理不同倾向。对应 OpenAI 的表述 “Humans steer. Agents execute.” 中的 steer。

2. 约束（Constrain）。防止执行超出安全范围。对应到 Agent：权限边界（误删文件需要被拦截）、Sandbox（隔离环境，执行出错也不影响本机）、Approval Gate（危险操作需要人工确认）、`Allowlist` / `denylist`（限定可操作的命令和路径），以及测试验证、rollback、audit log。

3. 整合（Integrate）。把各项能力配齐并协同。对应到 Agent：执行能力（tools、MCP、browser、filesystem）、状态承载（memory、logs）、协作机制（subagents、hooks、connectors）、自动化（CI、定时任务）。整合的作用在于配齐和协同：谁先调用谁、谁触发谁、谁的输出回传给谁。

这三类能力必须共同工作：只有引导没有约束，Agent 可能执行不该执行的动作；只有约束没有反馈，出错后无法修正；工具多但缺编排，长任务难以稳定完成。

业界三家的实践

在讨论 WorkBuddy 的做法之前，先看三家有代表性的实践：OpenAI、Anthropic 和 LangChain。

图：OpenAI 实践

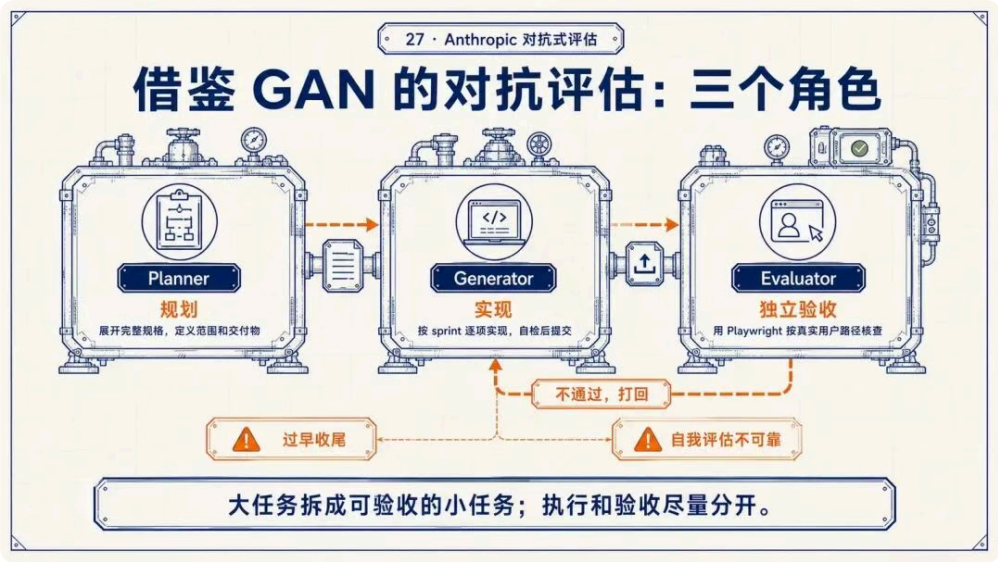
OpenAI：一个 3 人小组用 Codex 从空仓库开始开发，全程不手写代码，5 个月产出约 100 万行代码、1500 个 PR，并投入内外部使用。配套环境包括：让 Codex 直接操作浏览器、读取 DOM / 日志 / 监控指标；用 linter 和结构测试自动检查架构分层、依赖方向、命名和文件大小；把 `AGENTS.md` 改成目录入口、把详细知识结构化放进 docs 供按需查询；后台运行周期性任务扫描代码漂移、自动创建重构 PR。

这一案例说明在配套环境完整时，Agent 可以参与大规模代码生产。但其 Harness 主要集中在代码内部质量和可维护性上，没有说明功能和业务正确性如何维护。

Anthropic（第一篇：Effective harnesses for long-running agents）：处理长任务时发现两种典型失败：Agent 一次承担过多工作（中途耗尽上下文、给下一个 Agent 留下缺

少说明的半成品），或看到部分成果就过早判定完成。解决方案用“初始化 Agent + Coding Agent”两个角色完成跨会话交接：一开始就把要做的事拆成 200+ 条具体行为描述的功能清单（JSON）、每条标 pass/fail 并禁止删条目或降标准；一次只处理一项任务；统一的启动脚本 `init.sh`；除单测外用浏览器自动化做端到端验证；用进度文件 + Git 历史做交接、恢复和回滚。

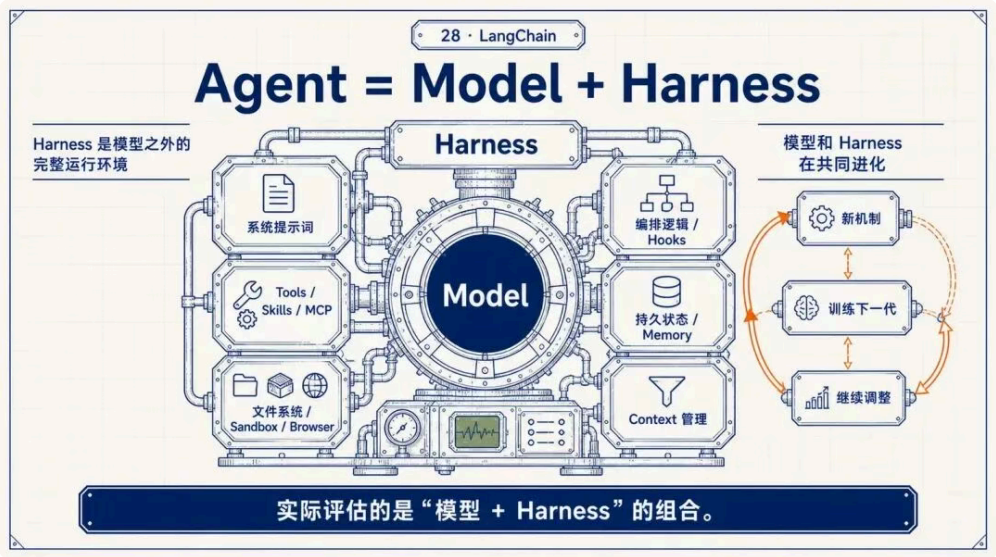
WorkBuddy 的 Agent 在这一点上做了类似设计：执行较大任务时，先把目标拆解成结构化任务清单，并在推进过程中持续更新状态。它同时处理“一次承担过多 / 过早判定完成”和“上下文遗忘”两类问题——显式任务状态可以让 Agent 在长对话里恢复进度，也让用户更容易判断任务是否真的完成。



图：跨会话任务交接

Anthropic（第二篇：Harness design for long-running application development）：在前一篇基础上发现两个更深的问题——接近上下文上限时模型会降低完成标准、自我评估不可靠（Agent 评价自己产出时倾向给正面结论，前端设计这类缺少确定性测试的任务尤其明显）。

方案借鉴 GAN 的对抗评估思路，用 Claude Agent SDK 构建三个角色：**Planner**（把一句话需求展开成完整规格，定范围但不指定实现细节）、**Generator**（按 sprint 逐功能实现、用 git 版本控制、提交前先自检）、**Evaluator**（独立验收 Agent，用 Playwright 像真实用户一样操作运行中的应用，逐条核查、把 bug 定位到行号和原因后打回）。普通用户可以借鉴的点：**分离执行和验收**（验收角色可用不同模型，或在 WorkBuddy 里用 Teams 分工）、**把标准写进规则文件、先确认需求再执行、随模型升级精简约束。**



图：Planner / Generator / Evaluator

LangChain：longchain 从框架构建者角度，把 Agent 中除模型之外的一切都纳入 Harness——系统提示词、Tools / Skills / MCP、文件系统、沙箱、浏览器、编排逻辑与 Hooks，这是一个更宽的定义：**Agent = Model + Harness**。它的几个核心观点：Agent 要有持久状态、能跨会话工作；无法为每件事预先准备工具，所以要给 Agent 一台计算机（Bash + 代码执行）；要能安全可扩展地运行代码、能自我验证；用 Compaction、Tool Call Offloading、Skills 渐进式加载等机制应对 Context Rot；用 Ralph Loop（Hook 拦截提前结束信号，在新上下文里重新注入目标继续执行）。

> *LangChain 还提出一个视角：模型和 Harness 在共同进化。实际评估的对象通常是“模型 + Harness”的组合——同一个模型换了工具名、Patch 格式、压缩策略或错误回传方式后，表现可能明显变化。因此比较或升级模型时，要把 Harness 一起纳入 Eval。*

WorkBuddy Agent 的五层 Harness（构建者视角）

WorkBuddy 构建良好的 Harness 有两个目标：**提高 Agent 首次执行的正确率，并提供反馈循环让系统能自动发现和纠正常见问题。**

在 WorkBuddy 的设计里，Harness 是一个控制系统：Agent 行动前，系统通过**前馈（Feedforward）**提供目标、规则、环境和可用能力；Agent 行动后，系统通过**反馈传感器（Feedback sensors）**观察结果，并把错误和修正信息返回给 Agent。

前馈提高第一次就做对的概率，反馈让 Agent 在问题进入人工审查前先自我纠正。只有前馈没有反馈，规则是否有效无法验证；只有反馈没有前馈，Agent 会反复犯同类错误，再依赖后置修正。

这两类控制还可以按执行方式分成**计算型和推断型。计算型**控制由确定性程序执行，例如 LSP、类型检查、linter、单元测试、结构测试、依赖扫描、脚本和 codemod，优点是快、便宜、可重复，适合在 Agent 每次修改后反复运行。**推断型**控制依赖模型做语义判断，例如 Review Agent、架构审查 Agent、AI judge 和设计评估，优点是能覆盖“是否过

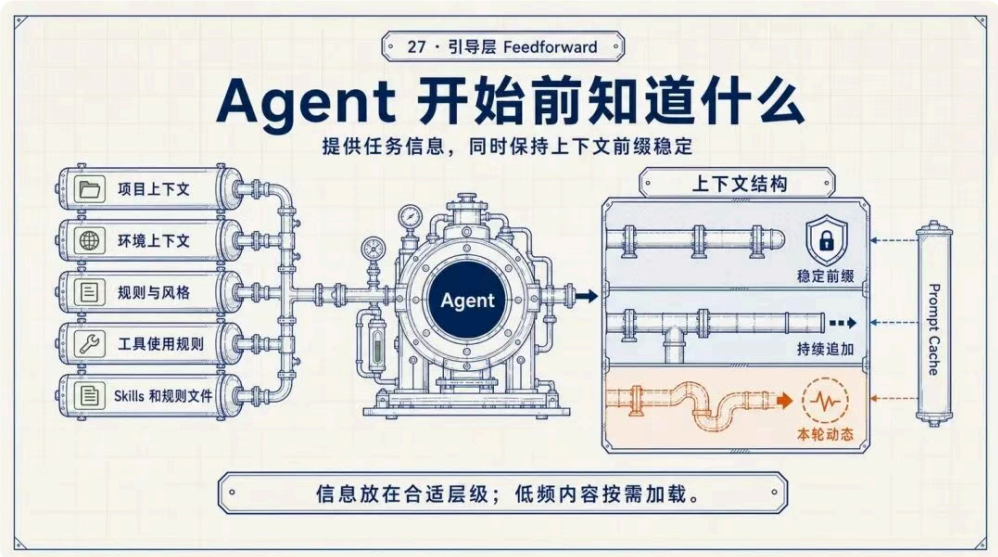
度设计”“是否误解需求”“是否符合团队约定”这类难以写成规则的问题，缺点是更慢、更贵，也更不确定。

WorkBuddy 的原则是：**能用计算型信号解决的问题，优先交给确定性程序；需要语义判断的问题，再交给审查 Agent。**反馈也要按时机分层：快速检查尽量左移到编辑后、提交前或 Agent 自我纠正循环中；更昂贵的架构审查、详细代码审查、端到端验证放到集成前后；持续漂移和运行时健康则交给周期性传感器，例如死代码扫描、覆盖率质量分析、依赖风险、延迟、错误率、可用性 SLO 和日志异常。这样 Harness 不只是一组规则，而是一套持续运行的控制系统。

图：Harness 五层结构

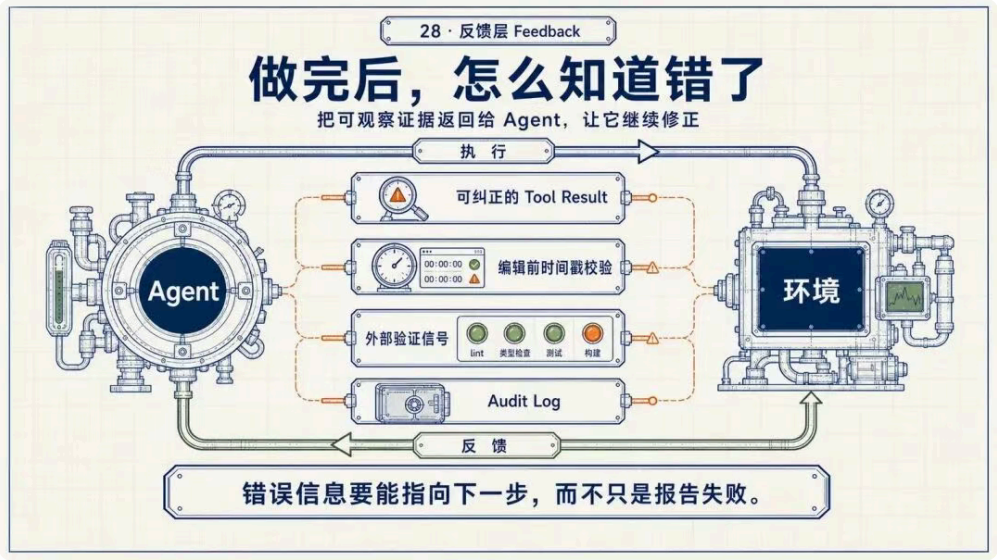
五个层次：

1. 运行环境层：Agent 在哪里执行。文件系统、Shell / Bash、Sandbox、Browser、MCP / Connectors、权限边界 / Approval Gate、`Allowlist` / `denylist`。这一层用户通常感知不到，但缺少任何一项，上面几层都难以稳定运行。LangChain 指出，文件系统是 Agent 最基础的运行环境，它支撑了持久状态、跨会话工作和多 Agent 协作。



图：引导层

2. 引导层（Feedforward）：Agent 开始前掌握什么。在执行前提供必要信息和约束，提高首次正确率。包括：项目上下文（项目概况、目录层级、关键依赖——早期模型不会主动探索代码库，可能在根目录写错文件）、环境上下文（操作系统、Shell、时间、时区、地理位置、IDE 主题、产品语言、已装 Skills、已连 Connectors）、规则与风格（不同模型不同倾向）、工具使用规则（独立的搜索 / 读取可并行、改文件前先读、路径不明先搜、长任务先拆 Todo）、Skills 和规则文件（把隐性知识保存成 Agent 能读到的内容）、上下文结构与 Prompt Cache（保持稳定前缀、追加动态内容、按需加载工具）。



图：反馈层

3. 反馈层（Feedback）：Agent 执行后如何获知错误。验证执行结果并把错误及修正信息返回给 Agent。工具结果包含可纠正信息（文件未找到提示搜索路径、编辑失败提示重新读取、权限不足提示请求确认、命令报错返回完整 stderr）——OpenAI 把 Agent 受阻视为“环境中缺少工具、规则或文档”的信号。编辑前的时间戳校验：Agent 改文件前比较“上次读取时间”和“文件最后修改时间”，若读取后被用户改过就拒绝写入、要求重读，避免覆盖最新修改。将外部验证信号返回 Agent：lint、类型检查、测试、构建等确定性信号成本低、可稳定重复；架构审查、代码审查和端到端验证这类推断型或高成本信号，则按风险放在更靠后的检查阶段。再加上 Audit log 让所有动作留痕、可追溯回放。

4. 编排层：多个能力如何组织。按任务组织能力、按需暴露上下文、为不同角色分配职责：渐进式加载、意图识别和路由、多模型路由、Teams 多 Agent 协作、并行工具调用。

5. 迭代层：Harness 自身如何持续调整。前面四层要随模型能力、用户场景和已发现的问题持续调整。WorkBuddy 一年内的几种迭代方式：

·**随模型能力提升精简上下文：**新模型会主动用 Glob / Grep 探索结构后，初始项目概况可以减少；

·**根据新问题增加约束：**输出过长、格式不稳定时补充更明确的行为边界；

·**针对不同模型适配工具：**根据实际表现调整工具组合；

·**根据重复出现的反馈增加机制：**反复出现文件覆盖、能力选择困难时补充写入保护、能力发现机制。

这类迭代需要证据支撑：一次失败可能是偶发，同类失败多次出现或风险很高时再调整 Harness。新增机制也要评估副作用：更严格的审批降低误操作风险，也增加打断；更多规则约束输出，也占用上下文。

WorkBuddy 团队作为使用者，如何建立 Harness

借用 OpenAI Codex 实验总结的四类组件，可以把 WorkBuddy 团队的现有实践归入同一框架。

图：使用者视角的四类组件

1. 上下文工程：让 Agent 获得当前任务所需的信息。分层规则文件（根目录 `AGENT.md` / `WORKBUDDY.md` 记录架构、依赖、安全和风格，子仓库用局部 `WORKBUDDY.md` 补充）、OpenSpec（较大变更先读写规范）、Skills（按任务加载）、Slash 命令（把高频流程组织为固定入口）。

2. 架构约束：把规则变成可执行检查。团队可以把架构规则、代码规范和提交要求接入本地检查、Git Hooks、CI 门禁和审查 Agent。确定性问题优先交给程序检查，需要语义判断的问题再交给审查 Agent。

3. 反馈循环：把验证结果返回给 Agent。Post-edit checkpoint（每次编辑后查行数、架构合规、注释完整性）、本地检查与 CI 结果、`/team:mr` 工作流（构建验证 → 生成 changeset → 创建 MR → 关联 Issue）、Dogfood Skill、Agent Browser。这一点对应 OpenAI 的表述：

> *“Agent 卡住时，我们把它当作信号——找出缺了什么（工具、护栏、文档），反哺回仓库——而且总是让 Codex 自己写这个修复。”*

4. 熵管理（Garbage Collection）：持续处理规则、代码和运行状态的漂移。现有检查主要防新增问题，对历史问题还要周期性扫描：`WORKBUDDY.md` / OpenSpec 与代码是否一致、历史代码是否违反新规则、是否有重复实现 / 失效文档 / 过期依赖。还可以接入运行时健康传感器，例如延迟、错误率、可用性 SLO、日志异常。

这四类组件形成一个持续过程：上下文工程提供规则和任务信息，架构约束阻止已知违规，反馈循环帮 Agent 修正本次执行，熵管理处理跨任务积累的漂移。

七、Loop Engineering：任务如何跨时间继续

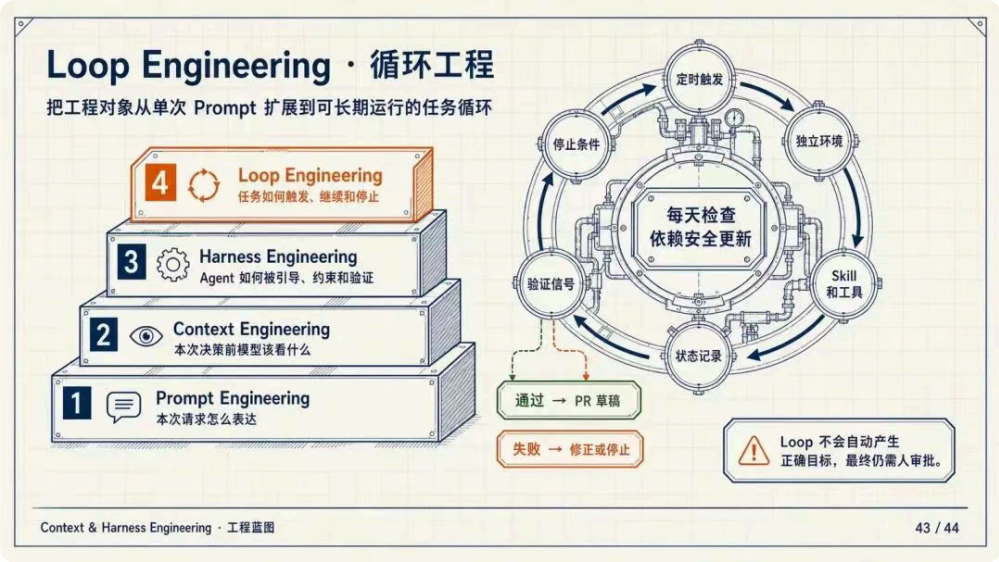
Loop Engineering 是近期出现的工程表达，目前还没有统一定义。前面几层关注“一次任务怎么做好”，Loop 关注 Agent 如何被触发、连续执行、验证结果、记录进度并再次运行——工程对象从单条 Prompt 扩展到可长期稳定运行的任务循环。

在四层工程中，Loop Engineering 的位置如下：

层次	核心问题	简单例子
Prompt Engineering	本次请求应如何表达？	写清目标、格式和约束
Context Engineering	这一次决策前，模型该看什么？	加载相关文件、工具、历史和记忆
Harness Engineering	Agent 如何被引导、约束、观测、验证和纠正？	规则、沙箱、审批、测试、日志、编排
Loop Engineering	任务如何被触发、流转、验收、继续与停止？	定时任务、工作树、子 Agent、记忆、反馈闭环

Harness 定义 Agent 的环境、权限、反馈和纠正机制；Loop 定义任务的触发、流转、重试和停止条件。Loop 依赖 Harness 提供的约束和验证机制，Harness 中的能力则可以通过 Loop 持续运行。

一个可用的 Loop 至少需要这些组件：触发器（Trigger / Automation）、独立执行环境（Isolated Workspace / Worktree）、Skills、Tools / Connectors / MCP、Sub-agents、Memory / Durable Artifacts、Sensors / Evals、Stop Conditions / Budget。



图：一个完整的 Loop

值得强调的是 Goal ≠ Loop：如果产品提供“设立长期目标”的能力，它确实能为 Loop 提供持久目标和进度载体，但 Goal 只定义“要去哪里”和“还剩什么”，Loop 还需要触发器、执行环境、工具、验证信号和停止条件。一个只会保存目标的功能，是 Loop 的状态组件，不是完整循环。

一个具体例子——每天检查依赖安全更新：

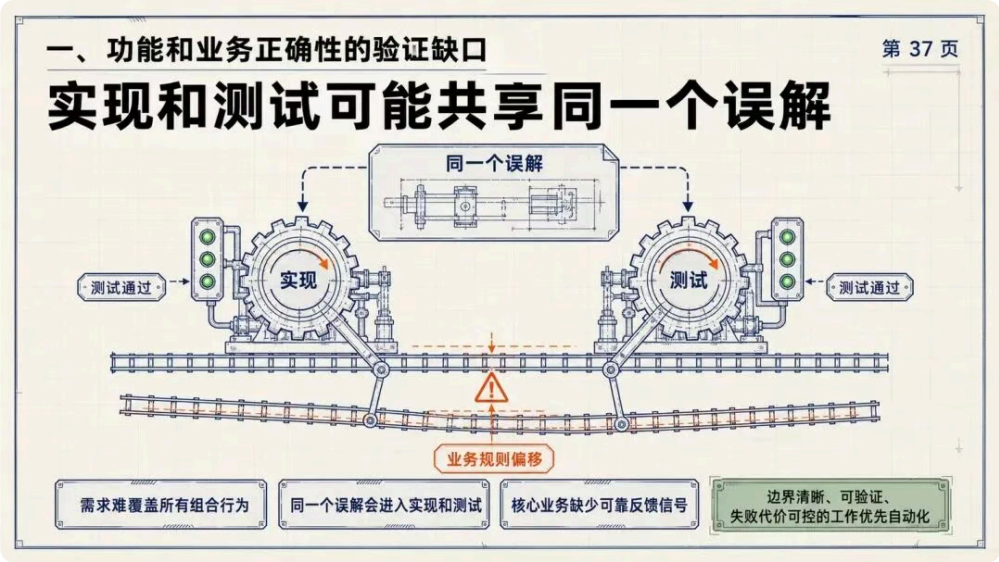
```
每天 09:00 定时触发。
创建独立 worktree 和任务记录。
读取仓库规则与 dependency-update Skill。
查询可用更新与已知漏洞。
选择一个可独立验证的更新，修改 lockfile。
运行安装、类型检查、单测、构建和漏洞扫描。
如果失败，将错误反馈给 Agent，在限定轮数内修正；仍失败则停止并留下可诊断报告。
如果通过，生成 PR 草稿和风险摘要，交由人审批发布。
写入本轮结果、未解问题和下次运行需要的交接信息。
```

但 Loop 不会自动解决以下问题：它不会自动产生正确目标（目标错误时，循环只会更快地朝错误方向执行）；不会自动产生可信的验收标准（Generator 和 Evaluator 若共享同一个误解，仍可能出现“错的实现 + 全部通过的测试”）；不会承担责任（发布、用户数据、支付、风控必须有明确的人类责任人和审批边界）；也不会替代工程师形成判断。

八、还没解决的问题

前面讨论的 Context、Memory、Harness 和 Loop，都是为了让 Agent 更稳定地完成任务。但这套体系仍有明确的能力边界，以下几个问题目前还没有成熟解法。

功能和业务正确性的验证缺口



图：业务正确性验证缺口

目前的 Harness 文章主要讨论架构分层、命名、复杂度、技术债和自动重构，对功能和业务正确性的验证讨论较少。缺少可规模化的业务验证方法，主要有四个原因：

需求本身很难完整说明。PRD 通常分别描述单项功能，不一定覆盖功能组合后的行为。例如一个会话同时支持“归档”和“置顶”，两个功能分别测试通过，组合后却出现新问题：已经置顶的会话能不能归档？如果 PRD 没有定义，Agent 可能自行决定，再把同一个理解同时写进实现和测试——工程检查全绿，但已加入一项未经确认的业务决策。

实现和测试可能共享同一个误解。同一个 Agent 既写实现又写测试，对需求的理解偏差会同时进两边。测试全过，仍不能证明实现符合原始业务意图。

部分业务正确性缺少可计算的判定标准。编译器能发现语法和类型问题，监控能度量性能和错误率，但业务意图通常需要业务人员确认。

业务错误的成本可能很高。核心业务错误可能造成资金损失、合规问题和用户流失，对验证可信度的要求远高于普通代码质量问题。

所以当业务正确性缺少可靠验证时，AI 的自治程度需要随风险提高而降低：

场景	AI 自治度上限
一次性脚本、内部工具	高
公开 API、跨系统改动	中
核心业务逻辑（支付、风控、订单）	低

判断一项工作是否适合交给 AI，可以先回答四个问题：有没有明确的完成标准？结果能不能用测试 / 规则 / 数据 / 人工审查验证？失败是否容易发现、可回滚、代价可控？任务是否重复发生、值得为它建 Harness？这四个问题共同指向**可验证性**。

代码库的 Harnessability 决定建设难度

问题严重的老系统通常更难建 Harness，原因有四：

1. Harness 依赖清晰的系统结构。明确边界、统一命名、稳定模块和有效测试，能让 Agent 更容易理解代码，老系统未必具备这些条件。

2. 大量历史违例会降低规则有效性。老 monorepo 一次可能产生数千个违例，全量修复成本高，大量白名单又会形成新的维护成本。

3. 高复杂度会影响 Agent 的理解能力。依赖关系复杂、隐式约定多时，Agent 更容易误判调用链、状态流转和副作用。

4. Harness 依赖可观测性和验证信号。老系统如果缺少埋点、指标口径不一致、链路追踪不完整，Agent 就只能依赖代码检查和自我评估，反馈层会明显变弱。

更可实现的做法：老系统先处理循环依赖和模块边界，同时补齐关键链路的测试、日志、指标和看板，再加 Harness；优先在一个结构清晰、修改频繁、价值高且可观测性较好的子模块验证方案再扩展；先约束新增和修改部分，再逐步处理存量。

案例有适用边界

目前主要案例来自模型厂商和框架团队。它们能证明 Harness 在特定环境的价值，但实验条件、可复现细节和业务验证方式并不完全公开。Harness 也需要和具体的代码库、技术栈、团队约定和可观测性水平一起设计，很难把某一套模板直接搬到所有系统。落到自己的代码库、模型和团队流程，仍要重新验证。

AI 可能推动技术方案标准化

以后选技术栈时，团队除了考虑性能、效率和生态，还会考虑它是否便于 AI 理解、修改和验证。即使两个方案都满足业务需求，团队也可能优先选已配好 Harness、结构统一的那个，因为 Agent 在其中工作更稳定。

有此可能就出现一种“Harness 模板”：围绕常见的服务拓扑，预先组合好结构约定、技术栈、指引和传感器。WorkBuddy 的 Service Template 已经在做类似的事。这不意味着所有项目都用同一种技术栈，更可能是团队减少缺乏维护的特殊方案，把投入集中到几套验证完整的标准方案上。

Harness 需要持续投入

OpenAI 那个实验从空仓库到 100 万行代码用了 5 个月，文章里明确说：

> *“this isn't something you can jump into for quick results.” / “Our most difficult challenges now center on designing environments, feedback loops, and control systems.”*

这和 Chad Fowler 的“Relocating Rigor”一致：工程严谨度从代码编写本身，部分转移到环境、反馈回路和控制系统的设计上。规则要更新、测试要补充、工具和 Skill 要适配新模型、历史代码要持续治理——Harness 是工程基础设施，不是一次性配置。

人仍然负责主线任务

Harness 应优先覆盖重复、确定、可验证的工作；探索和业务判断的工作应由人主导。AI 可以提高执行效率，但团队仍需保持对代码、逻辑和架构的理解，并在系统出问题时能定位原因。借 Addy Osmani 的话：

> *“The danger is stopping having an opinion when loops run autonomously.”*

同样的 Loop 可以加深对工作的理解，也可能让人逐渐不再关注过程。Harness 的效果仍然依赖人的目标设定、验收标准和最终判断。

总结

1.模型 是核心推理与生成引擎，但它不会自动保留状态、不会自动联网、不会自动执行外部动作，当前任务的材料须由产品注入。

2.Function Call/MCP/Skill/Plugin 构成能力层：让模型请求动作、接入外部系统、沉淀任务流程、打包分发能力。

3.Context Engineering 决定模型当前能看到什么；**Memory**决定哪些过去可以在未来重新出现。

4.Harness Engineering 把指引、能力、权限、反馈、验证与编排组成可信系统，通过前馈和反馈提高可靠性。

5.Loop Engineering 把这套系统放进时间维度，让任务能被反复触发、检验、交接和收敛。

两个核心结论

模型决定能力上限；上下文和 Harness 决定这个上限能否稳定落地。

人负责选择方向、定义标准并承担责任；Agent 负责执行、验证和加速迭代。

欢迎前往官网下载体验
亲自感受更多惊喜功能🎉

www.workbuddy.cn

腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容

公众号

阅读 1.7万

留言 1

写留言

2025 河北 7月24日 12

我最近发现一个AI工具叫做WeChat，里面好多智能体，聊天都是免费的，识图能力还可以。就是推理速度有点慢，经常会重复你的输入。

首评